



TRƯỜNG ĐẠI HỌC QUY NHƠN

TIỂU LUẬN HỌC PHẦN
LÝ THUYẾT TỐI ƯU

THUẬT TOÁN DI TRUYỀN:
CƠ SỞ LÝ THUYẾT VÀ CÁC
ỨNG DỤNG

Trình độ đào tạo:	Thạc sĩ
Học viên thực hiện:	Nguyễn Hồ Bảo Thiên
Lớp:	Khoa học dữ liệu K28B
Giảng viên hướng dẫn:	TS. Nguyễn Văn Vũ

Gia Lai, 2026

Mục lục

1	Cơ sở lý thuyết về Thuật toán Di truyền	4
1.1	Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa	4
1.2	Cơ sở Khoa học và Nguyên lý Hoạt động của GA	6
1.3	Các Thành phần và Phép toán Cốt lõi	7
1.4	Quy trình Thuật toán và Điều kiện Dừng	11
	Tài liệu tham khảo	14

Danh sách hình vẽ

1.1	Chu trình tiến hóa của Thuật toán Di truyền	12
-----	---	----

Danh sách bảng

1.1	Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền	7
-----	---	---

Chương 1

Cơ sở lý thuyết về Thuật toán Di truyền

1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa

Bài toán Tối ưu hóa và Không gian Tìm kiếm

Một bài toán tối ưu hóa tổng quát yêu cầu tìm một vector biến quyết định $x = (x_1, x_2, \dots, x_n)$ sao cho một hàm mục tiêu (objective function) $f(x)$ đạt giá trị cực tiểu hoặc cực đại, trong khi x vẫn thỏa mãn một tập ràng buộc xác định miền khả thi (feasible region) Ω :

$$\min_{x \in \Omega} f(x) \quad \text{hoặc} \quad \max_{x \in \Omega} f(x). \quad (1.1)$$

Miền $\Omega \subseteq \mathbb{R}^n$, còn gọi là không gian tìm kiếm (search space), thường được xác định bởi một hệ ràng buộc đẳng thức và bất đẳng thức trên các biến quyết định. Bài toán được coi là **có ràng buộc** (constrained) khi Ω là một tập con thực sự của không gian biến, và **không ràng buộc** (unconstrained) khi Ω trùng với toàn bộ không gian biến.

Hai thách thức lớn khiến tối ưu hóa trở thành một bài toán khó nói chung là:

- **Cực trị cục bộ và cực trị toàn cục.** Khi hàm mục tiêu không lồi (non-convex) hoặc đa cực trị (multimodal), một thuật toán chỉ dựa vào thông tin cục bộ quanh điểm hiện tại — ví dụ đạo hàm — có xu hướng hội tụ về cực trị cục bộ (local optimum) gần điểm khởi tạo nhất, thay vì cực trị toàn cục (global optimum) của toàn miền Ω . Đây chính là hạn chế cốt lõi của các phương pháp tối ưu dựa trên gradient khi áp dụng cho bài toán không lồi.
- **Bài toán NP-hard và sự bùng nổ tổ hợp.** Với nhiều lớp bài toán tối ưu tổ hợp (ví dụ bài toán người bán hàng — TSP, bài toán lập lịch), số lượng phương án khả thi tăng theo hàm mũ hoặc giai thừa của kích thước bài toán, khiến việc

duyet toàn bộ không gian tìm kiếm trở nên bất khả thi về mặt tính toán ngay cả với các bài toán kích thước vừa phải.

Hai thách thức trên là động lực chính cho sự phát triển của các phương pháp tìm kiếm không dựa trên đạo hàm, có khả năng khám phá đồng thời nhiều vùng của không gian tìm kiếm — trong đó có họ thuật toán tiến hóa được trình bày ở mục tiếp theo.

Họ Thuật toán Tiến hóa (Evolutionary Computation)

Khác với các phương pháp tối ưu cổ điển dựa trên đạo hàm (ví dụ Gradient Descent, phương pháp Newton) chỉ xử lý một điểm duy nhất tại mỗi bước lặp, **metaheuristic** là các chiến lược tìm kiếm ở mức tổng quát, không đảm bảo tìm được nghiệm tối ưu toàn cục tuyệt đối nhưng thường tìm được nghiệm đủ tốt trong thời gian chấp nhận được. Metaheuristic đặc biệt hữu ích khi hàm mục tiêu không khả vi, có nhiễu, không lồi, hoặc thậm chí không có biểu thức giải tích tường minh (black-box function).

Một nhánh quan trọng của metaheuristic là các **thuật toán tìm kiếm dựa trên quần thể** (population-based search): thay vì duy trì một điểm duy nhất, thuật toán duy trì một tập hợp nhiều điểm (quần thể) và để chúng cùng tiến hóa qua các thế hệ, nhờ đó khảo sát đồng thời nhiều vùng của không gian tìm kiếm và giảm nguy cơ mắc kẹt tại một cực trị cục bộ duy nhất. **Họ Thuật toán Tiến hóa** (Evolutionary Algorithms — EA) là đại diện tiêu biểu nhất của nhóm này, mô phỏng nguyên lý chọn lọc tự nhiên của Darwin để dẫn dắt quá trình tìm kiếm. Họ EA bao gồm bốn nhánh chính, phân biệt nhau chủ yếu ở cách biểu diễn giải pháp và toán tử tiến hóa được nhấn mạnh:

- **Thuật toán Di truyền (Genetic Algorithm — GA):** nhấn mạnh vai trò của lai ghép (crossover) như toán tử tìm kiếm chính, kết hợp thông tin từ nhiều cá thể cha mẹ để tạo ra cá thể con; đây là đối tượng nghiên cứu chính của chương này.
- **Chiến lược Tiến hóa (Evolution Strategies — ES):** do Rechenberg và Schwefel phát triển, tập trung vào tối ưu tham số thực với đột biến Gauss là toán tử chính và có cơ chế tự thích nghi (self-adaptation) các tham số đột biến trong quá trình tiến hóa.
- **Lập trình Tiến hóa (Evolutionary Programming — EP):** do Fogel đề xuất, tiến hóa chủ yếu thông qua đột biến trên biểu diễn hành vi (phenotype) của cá thể, gần như không sử dụng lai ghép.
- **Lập trình Di truyền (Genetic Programming — GP):** do Koza phát triển, mở rộng GA để tiến hóa trực tiếp các cấu trúc cây biểu thức hoặc chương trình

máy tính, thường dùng để tự động khám phá công thức toán học hoặc chương trình giải quyết một tác vụ cho trước.

Trong phạm vi tiểu luận này, trọng tâm là Thuật toán Di truyền áp dụng cho bài toán tối ưu số thực có ràng buộc, với cách mã hóa và các toán tử cụ thể được trình bày trong các mục tiếp theo.

1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA

Nguồn gốc Lịch sử và Nguyên lý Sinh học

Thuật toán Di truyền được John Holland giới thiệu năm 1975 tại Đại học Michigan trong công trình *Adaptation in Natural and Artificial Systems* [1], và sau đó được David Goldberg hệ thống hoá, phổ biến rộng rãi hơn qua cuốn sách kinh điển năm 1989 *Genetic Algorithms in Search, Optimization, and Machine Learning* [2]. Ý tưởng nền tảng của GA là mô phỏng ba nguyên lý cốt lõi trong học thuyết tiến hóa của Darwin:

- **Chọn lọc tự nhiên** (*"Survival of the Fittest"*): cá thể thích nghi tốt hơn với môi trường có xác suất sống sót và sinh sản cao hơn.
- **Di truyền** (Heredity): đặc điểm của cá thể cha mẹ được truyền lại cho thế hệ con thông qua vật chất di truyền.
- **Biến dị** (Variation): các sai khác ngẫu nhiên xuất hiện giữa các cá thể — do lai ghép và đột biến — là nguồn tạo ra tính đa dạng cần thiết để quá trình chọn lọc có tác dụng.

GA chuyển ba nguyên lý sinh học trên thành một quy trình tính toán bằng cách xây dựng một phép tương ứng giữa các khái niệm sinh học và các thành phần thuật toán, được tóm tắt trong Bảng 1.1.

Nhờ phép tương ứng này, một bài toán tối ưu bất kỳ có thể được diễn dịch lại thành một quá trình "tiến hóa nhân tạo": xuất phát từ một quần thể giải pháp ngẫu nhiên, để các giải pháp tốt hơn có nhiều cơ hội "sinh sản" hơn qua nhiều thế hệ, quần thể dần dịch chuyển về phía các vùng có giá trị hàm mục tiêu tốt của không gian tìm kiếm.

Phương pháp Mã hóa Giải pháp (Representation / Encoding)

Bước đầu tiên khi áp dụng GA cho một bài toán cụ thể là chọn cách biểu diễn (mã hóa) một giải pháp dưới dạng "nhiễm sắc thể" mà các toán tử di truyền có thể thao tác được. Bốn cách mã hóa phổ biến nhất là:

Bảng 1.1: Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền

Sinh học	Thuật toán Di truyền (GA)
Nhiễm sắc thể (Chromosome)	Mã hóa giải pháp (Solution encoding)
Gen (Gene)	Một biến / giá trị thành phần trong giải pháp
Cá thể (Individual)	Một giải pháp ứng viên
Quần thể (Population)	Tập hợp các giải pháp tại một thế hệ
Sức sống / Độ thích nghi	Giá trị hàm độ thích nghi (Fitness score)
Thế hệ (Generation)	Một vòng lặp tiến hóa

Mã hóa Nhị phân (Binary Encoding): giải pháp được biểu diễn bằng một chuỗi bit 0 và 1, ví dụ 10110010. Đây là cách mã hóa nguyên thủy nhất trong GA cổ điển của Holland, phù hợp với các bài toán rời rạc hoặc logic, nhưng khi áp dụng cho biến số thực cần một bước rời rạc hóa (lượng tử hóa) làm giảm độ chính xác.

Mã hóa Số thực (Real-value Encoding): giải pháp là một dãy số thực, ví dụ $[3.14, -0.05, 12.8]$, mỗi phần tử tương ứng trực tiếp với một biến quyết định. Cách mã hóa này phù hợp tự nhiên với các bài toán tối ưu hóa liên tục nhiều biến, tránh được sai số lượng tử hóa của mã hóa nhị phân, và là cách mã hóa được sử dụng trong phần cài đặt thực nghiệm của tiểu luận (Chương 2).

Mã hóa Hoán vị (Permutation Encoding): giải pháp là một thứ tự sắp xếp của một tập hợp phần tử, ví dụ $[3, 1, 4, 2, 5]$. Đây là cách mã hóa tự nhiên cho các bài toán định tuyến/lập lịch như bài toán người bán hàng (TSP) hay bài toán phân công công việc (Scheduling), nơi lời giải về bản chất là một hoán vị.

Mã hóa Dựa trên Cấu trúc (Tree/Graph Encoding): giải pháp được biểu diễn dưới dạng cây biểu thức hoặc đồ thị, sử dụng chủ yếu trong Lập trình Di truyền (Genetic Programming) để tiến hóa các công thức hoặc chương trình.

Việc lựa chọn cách mã hóa quyết định trực tiếp các toán tử lai ghép và đột biến nào có thể áp dụng được, như sẽ trình bày ở Mục 1.3.

1.3 Các Thành phần và Phép toán Cốt lõi

Hàm độ thích nghi (Fitness Function) và Ràng buộc

Hàm độ thích nghi (fitness function) đóng vai trò đánh giá chất lượng của từng cá thể trong quần thể, làm cơ sở cho phép chọn lọc. Với bài toán tối ưu không ràng buộc, hàm thích nghi thường được xây dựng trực tiếp từ hàm mục tiêu (objective function), ví

dự đảo dấu hàm mục tiêu khi bài toán gốc là bài toán cực tiểu hóa nhưng GA nguyên thủy được phát biểu dưới dạng cực đại hóa độ thích nghi.

Đối với bài toán **có ràng buộc** — trọng tâm của tiểu luận này — cách tiếp cận kinh điển là chuyển ràng buộc thành một số hạng phạt (penalty) cộng vào hàm mục tiêu, tạo thành hàm thích nghi đã phạt:

$$F(x) = f(x) \pm \sum_i \lambda_i \cdot g_i(x), \quad (1.2)$$

trong đó $g_i(x)$ đo mức độ vi phạm ràng buộc thứ i (bằng 0 nếu ràng buộc được thỏa mãn), còn $\lambda_i > 0$ là hệ số phạt tương ứng; dấu $+$ hay $-$ và độ lớn của λ_i được chọn tùy theo bài toán là cực tiểu hay cực đại hóa, sao cho cá thể vi phạm ràng buộc luôn bị đánh giá kém hơn cá thể khả thi có cùng giá trị hàm mục tiêu.

Hạn chế cố hữu của hàm phạt tĩnh (static penalty) là hệ số λ_i phải được chọn thủ công và cố định trong suốt quá trình tiến hóa: chọn λ_i quá nhỏ khiến quần thể "phớt lờ" ràng buộc, trong khi chọn quá lớn khiến số hạng phạt lấn át hoàn toàn hàm mục tiêu, làm quần thể chỉ tập trung thỏa mãn ràng buộc mà mất khả năng phân biệt các nghiệm khả thi tốt và xấu. Nhiều phương pháp xử lý ràng buộc nâng cao hơn đã được đề xuất để khắc phục hạn chế này — ví dụ quy tắc khả thi của Deb (feasibility rules) hay các ngưỡng ε giảm dần theo thời gian (Takahama và Sato) — và sẽ được trình bày cụ thể cùng phần cài đặt thực nghiệm ở Chương 2.

Các Phép chọn lọc (Selection Operators)

Phép chọn lọc quyết định cá thể nào trong quần thể hiện tại được chọn làm "bố mẹ" để sinh ra thế hệ tiếp theo, thiên vị có chủ đích về phía các cá thể có độ thích nghi cao hơn.

Vòng quay Roulette (Roulette Wheel Selection): xác suất chọn cá thể i tỉ lệ thuận với độ thích nghi f_i của nó so với tổng độ thích nghi toàn quần thể:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}. \quad (1.3)$$

Hạn chế của phương pháp này là khi một cá thể có độ thích nghi vượt trội hẳn phần còn lại (super-individual), nó sẽ chiếm phần lớn xác suất được chọn, khiến quần thể nhanh chóng mất đa dạng.

Thách đấu (Tournament Selection): chọn ngẫu nhiên k cá thể từ quần thể, sau đó cá thể tốt nhất trong nhóm k cá thể này được chọn làm bố/mẹ. Tham số k (kích thước thách đấu) điều khiển trực tiếp áp lực chọn lọc: k càng lớn, áp lực

chọn lọc càng cao và quần thể hội tụ càng nhanh, nhưng cũng dễ mất đa dạng hơn.

Xếp hạng (Rank-based Selection): cá thể được chọn dựa trên thứ hạng của nó trong quần thể (sau khi sắp xếp theo độ thích nghi) thay vì giá trị thích nghi tuyệt đối, giúp tránh hiện tượng một cá thể siêu việt áp đảo toàn bộ xác suất chọn như ở Roulette Wheel.

Bên cạnh các toán tử chọn lọc bố mẹ nêu trên, một chiến lược lựa chọn cá thể sang thế hệ sau đặc biệt quan trọng là **giữ lại cá thể ưu tú (Elitism)**, được trình bày riêng ở phần dưới đây do vai trò và cách áp dụng của nó khác với chọn lọc bố mẹ thông thường.

Các Phép lai ghép (Crossover Operators)

Lai ghép (crossover) kết hợp vật chất di truyền của hai cá thể bố mẹ để tạo ra cá thể con, với tần suất lai ghép p_c (crossover rate) thường nằm trong khoảng 0.6 đến 0.95 — nghĩa là phần lớn, nhưng không phải toàn bộ, các cặp bố mẹ được chọn sẽ thực sự lai ghép; phần còn lại được sao chép nguyên vẹn sang thế hệ con.

Đối với mã hóa nhị phân và số thực, ba toán tử phổ biến nhất là:

- **Lai ghép đơn điểm (Single-point Crossover):** chọn ngẫu nhiên một vị trí cắt trên chuỗi nhiễm sắc thể, hai cá thể con được tạo bằng cách hoán đổi các đoạn nằm sau vị trí cắt giữa hai bố mẹ.
- **Lai ghép đa điểm (Multi-point Crossover):** tổng quát hóa lai ghép đơn điểm với nhiều vị trí cắt, các đoạn xen kẽ giữa hai bố mẹ được hoán đổi.
- **Lai ghép đồng nhất (Uniform Crossover):** mỗi vị trí gen được quyết định độc lập, ngẫu nhiên xem sẽ lấy từ bố hay mẹ, không phụ thuộc vào vị trí trong chuỗi.

Với mã hóa số thực, một họ toán tử quan trọng khác là lai ghép trộn (blend crossover, BLX- α) — cách tiếp cận được sử dụng trong phần cài đặt của tiểu luận. Với hai cá thể bố mẹ $p_1, p_2 \in \mathbb{R}^n$ và một hệ số trộn α được lấy mẫu ngẫu nhiên (có thể vượt ra ngoài khoảng $[0, 1]$), hai cá thể con được tạo theo tổ hợp tuyến tính:

$$c_1 = \alpha \cdot p_1 + (1 - \alpha) \cdot p_2, \quad c_2 = \alpha \cdot p_2 + (1 - \alpha) \cdot p_1. \quad (1.4)$$

Khi α được lấy mẫu từ một khoảng mở rộng ra ngoài $[0, 1]$ (ví dụ $\alpha \sim \mathcal{U}(-0.25, 1.25)$) thay vì giới hạn trong đoạn nối hai bố mẹ, cá thể con có thể nằm ngoài đoạn thẳng nối p_1 và p_2 , giúp toán tử này vừa có khả năng khai thác (exploitation) vùng lân cận

bố mẹ, vừa giữ được khả năng khám phá (exploration) ra ngoài đoạn đó thay vì chỉ co dần khoảng tìm kiếm qua các thế hệ.

Đối với mã hóa hoán vị (bài toán định tuyến/lập lịch), việc hoán đổi trực tiếp từng đoạn như trên có thể tạo ra cá thể con vi phạm tính hoán vị (một phần tử xuất hiện nhiều lần hoặc bị thiếu), nên cần các toán tử chuyên biệt:

- **PMX (Partially Matched Crossover)**: hoán đổi một đoạn con giữa hai bố mẹ như lai ghép đơn điểm, sau đó sửa các vị trí còn lại theo một ánh xạ tương ứng từng phần để đảm bảo tính hoán vị.
- **OX (Order Crossover)**: giữ nguyên một đoạn con từ bố mẹ thứ nhất, các phần tử còn lại được điền theo đúng thứ tự xuất hiện của chúng ở bố mẹ thứ hai.
- **CX (Cycle Crossover)**: xác định các "chu trình"(cycle) giữa vị trí phần tử ở hai bố mẹ, mỗi cá thể con được tạo bằng cách giữ nguyên các chu trình xen kẽ từ mỗi bố mẹ.

Các Phép đột biến (Mutation Operators)

Đột biến (mutation) tạo ra biến dị ngẫu nhiên trên một cá thể, với tần suất đột biến p_m (mutation rate) trên mỗi gen thường nhỏ, nằm trong khoảng 0.001 đến 0.05 đối với GA nhị phân cổ điển; vai trò chính của đột biến là duy trì đa dạng di truyền và giúp thuật toán thoát khỏi các vùng mà lai ghép một mình không thể tiếp cận được.

- **Đột biến đảo bit (Bit-flip Mutation)**: với mã hóa nhị phân, mỗi bit được đảo giá trị ($0 \leftrightarrow 1$) độc lập với xác suất p_m .
- **Đột biến Gauss / Uniform**: với mã hóa số thực, mỗi biến x_i được cộng thêm một nhiễu ngẫu nhiên, phổ biến nhất là nhiễu Gauss:

$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2), \quad (1.5)$$

trong đó độ lệch chuẩn σ_i thường được đặt tỉ lệ với độ rộng miền giá trị của biến x_i , sao cho bước đột biến có ý nghĩa tương đối giống nhau giữa các biến có thang đo khác nhau.

- **Đột biến hoán đổi, đảo ngược, chèn** (Swap, Inversion, Insertion): dành cho mã hóa hoán vị — lần lượt là hoán đổi vị trí hai phần tử, đảo ngược thứ tự một đoạn con, và di chuyển một phần tử sang vị trí khác trong chuỗi — mỗi thao tác đều bảo toàn tính hoán vị của cá thể.

Chiến lược giữ lại Cá thể Ưu tú (Elitism)

Vì chọn lọc, lai ghép và đột biến đều mang tính ngẫu nhiên, không có gì đảm bảo cá thể tốt nhất của thế hệ hiện tại sẽ xuất hiện lại ở thế hệ kế tiếp — trên lý thuyết, độ thích nghi tốt nhất toàn quần thể hoàn toàn có thể giảm đi giữa hai thế hệ liên tiếp. **Elitism** khắc phục vấn đề này bằng cách bắt buộc giữ lại nguyên vẹn k cá thể tốt nhất của thế hệ hiện tại, chuyển thẳng sang thế hệ sau mà không qua lai ghép hay đột biến. Nhờ đó, độ thích nghi tốt nhất từng tìm được không bao giờ giảm qua các thế hệ (đơn điệu không giảm). Việc chọn kích thước elitism k là một sự đánh đổi: k quá nhỏ có nguy cơ đánh mất cá thể tốt vào tay các toán tử ngẫu nhiên, còn k quá lớn làm giảm áp lực khám phá của quần thể, có thể góp phần làm giảm đa dạng của quần thể và khiến quá trình tìm kiếm hội tụ sớm về một vùng hẹp của không gian tìm kiếm.

1.4 Quy trình Thuật toán và Điều kiện Dừng

Sơ đồ khối và Giả mã (Pseudocode)

Kết hợp các thành phần đã trình bày ở Mục 1.3, một thể hệ GA điển hình lặp lại chu trình: chọn lọc bố mẹ, lai ghép, đột biến, đánh giá quần thể con, rồi cập nhật quần thể mới có áp dụng elitism — như minh họa ở Hình 1.1.

Thuật toán 1 trình bày lại chu trình này dưới dạng giả mã.

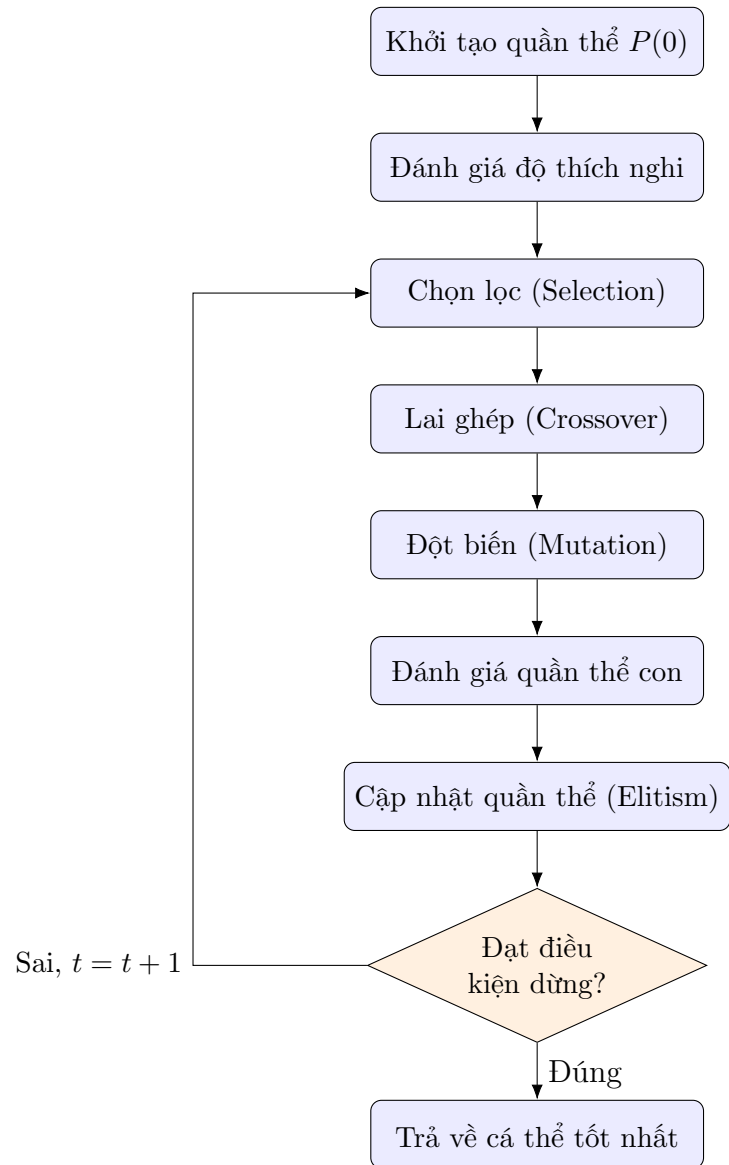
Thuật toán 1 Giải mã tổng quát của Thuật toán Di truyền

```
1:  $t \leftarrow 0$ 
2: Khởi tạo quần thể ban đầu  $P(0)$  ngẫu nhiên
3: Đánh giá độ thích nghi của  $P(0)$ 
4: while điều kiện dừng chưa thỏa mãn do
5:    $t \leftarrow t + 1$ 
6:   Chọn lọc tập bố mẹ  $S(t)$  từ  $P(t - 1)$ 
7:   Thực hiện lai ghép (crossover) trên  $S(t)$ , tạo tập con  $C(t)$ 
8:   Thực hiện đột biến (mutation) trên  $C(t)$ 
9:   Đánh giá độ thích nghi của  $C(t)$ 
10:   $P(t) \leftarrow$  cập nhật quần thể mới (áp dụng elitism)
11: end while
12: return cá thể tốt nhất tìm được
```

Tiêu chuẩn Dừng và Đánh giá sự Hội tụ

Vòng lặp tiến hóa cần một điều kiện dừng (termination criterion) tường minh; ba tiêu chuẩn phổ biến, có thể kết hợp với nhau, là:

- **Đạt số lượng thế hệ tối đa G_{max} :** đơn giản và dễ kiểm soát ngân sách tính toán nhất, nhưng G_{max} quá nhỏ có nguy cơ dừng trước khi quần thể hội tụ, trong



Hình 1.1: Chu trình tiến hóa của Thuật toán Di truyền

khi G_{max} quá lớn gây lãng phí tính toán không cần thiết sau khi quần thể đã hội tụ.

- **Đạt giá trị fitness mục tiêu:** dừng ngay khi tìm được một cá thể có độ thích nghi đạt hoặc vượt một ngưỡng đặt trước — phù hợp khi đã biết (hoặc ước lượng được) giá trị tối ưu cần đạt.
- **Quần thể hội tụ (stagnation):** dừng khi độ thích nghi của cá thể tốt nhất không cải thiện sau K thế hệ liên tiếp — dấu hiệu cho thấy các toán tử di truyền không còn tạo ra tiến triển đáng kể. Cần lưu ý tiêu chuẩn này có thể dừng thuật toán quá sớm nếu K quá nhỏ, do quần thể có thể dừng cải thiện tạm thời trước khi tiếp tục tiến triển ở các thế hệ sau.

Trên cơ sở các thành phần và quy trình đã trình bày ở chương này, Chương 2 sẽ trình bày cụ thể việc cài đặt một GA mã hóa số thực cho bài toán tối ưu có ràng buộc, đồng thời so sánh thực nghiệm với phương pháp SciPy SLSQP. Độc giả quan tâm đến các hướng phát triển và ứng dụng khác của GA có thể tham khảo thêm tổng quan của Katoch và cộng sự (2020) [5], cũng như một ứng dụng kinh điển kết hợp GA với huấn luyện mạng nơ-ron của Whitley và cộng sự (1990) [6].

Tài liệu tham khảo

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press.
4. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
5. Katoch, S., Chauhan, S. S., & Kumar, V. (2020). A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, 80, 8091–8126.
6. Whitley, D., Starkweather, T., & Bogart, C. (1990). Genetic algorithms and neural networks: optimizing connections and connectivity. *Parallel Computing*, 14, 347–361.
7. Takahama, T., & Sato, S. (2010). Constrained optimization by the ε constrained differential evolution with gradient-based mutation and feasible elites. *IEEE Congress on Evolutionary Computation (CEC)*.